

LLM-MODUL 3

DAS GEDÄCHTNIS DES RUNENSTEINS

CONTEXT & MEMORY





BJÖRN CYBERVIKING

— SI-LLM-03 —
**DAS GEDÄCHTNIS
DES RUNENSTEINS**
Context & Memory – Nur was auf dem Tisch liegt, zählt.



WISSEN
MACHT
KLARERE
FRAGEN

1 DIE INFORMATION

Orakel, mein Schiff
heißt Sturmfalke.

Sturmfalke.
Verstanden.

STURMFALKE

2 AUF DEM TISCH

Wie viele Männer
sind an Bord?

Das hast du mir
noch nicht erzählt.

STURMFALKE

3 DER CONTEXT

Dann erzähle ich dir mehr.
Drei Masten, zwölf Mann
Besatzung und morgen
geht es nach Norden.

Jetzt habe ich
diese Informationen
vor mir.

STURMFALKE

DREI
MASTEN

ZWÖLF
MANN

NACH
NORDEN

4 DIE INFORMATIONEN VERBINDEN SICH



Du kannst also
alles miteinander
verbinden?

Solange diese
Informationen
vor mir liegen.

5 ABER IM LANGHAUS LIEGT NOCH MEHR

Aber dort liegen
doch noch viel mehr
Schriftrollen!

Ja. Aber sie liegen
nicht auf meinem
Tisch.

STURMFALKE

6 DER TISCH IST VOLL

Dann legen wir einfach
noch diese hier dazu.

Mein Tisch
hat Grenzen,
Björn.

7 DER CONTEXT WIRD VERWALTET

Wenn der Tisch voll wird,
müssen Informationen
gekürzt oder zusammen-
gefasst werden.

ZUSAMMENFASSUNG

- Sturmfalke
- 3 Masten
- 12 Mann
- Reise nach Norden

8 EINE INFORMATION KOMMT ZURÜCK

Und was war noch
einmal mit dem Hafen?

Jetzt kann ich diese
Information wieder
berücksichtigen.

HAFEN

9 DIE ERKENNTNIS

Jetzt verstehe ich.
Du hast also nicht
einfach alles im Kopf.

Ich arbeite mit dem,
was gerade vor mir
liegt.

MERKE:

Ein Large Language Model (LLM) hat kein dauerhaftes Gedächtnis im klassischen Sinn. Es verarbeitet die Informationen, die ihm im aktuellen Context zur Verfügung stehen. Die Größe dieses Contexts ist begrenzt. Weitere Informationen können außerhalb gespeichert und bei Bedarf wieder in den Context eingebracht werden.



= LLM (Orakelwiking)



= Context (Schriftrollen auf dem Tisch)



= Context Window (Größe des Tisches)



= Memory (gespeicherte Schriftrollen)



BJÖRN
CYBERVIKING

GROSSE FRAGEN.
EINFACH ERKLÄRT.

— LLM-MODUL 3 — DAS GEDÄCHTNIS DES RUNENSTEINS CONTEXT & MEMORY

WISSEN
VERSTEHEN
ZUSAMMEN-
HÄNGE
ERKENNEN

1

CONTEXT – WAS LIEGT AUF DEM TISCH?

Context ist die Menge an Informationen, die dem Modell für die aktuelle Verarbeitung zur Verfügung stehen.

Hier sind die wichtigsten Informationen zu meinem Schiff, Orakel.

Diese Informationen habe ich jetzt vor mir.

NICHT ALLES, WAS EXISTIERT, LIEGT GERADE AUF DEM TISCH.

Nur die Informationen, die auf dem Tisch liegen, kann das Modell in dieser Antwort berücksichtigen.

STURMFALKE

3 MASTEN

12 MANN

NACH NORDEN

2

CONTEXT WINDOW – DER TISCH HAT GRENZEN

Das Context Window ist die begrenzte Menge an Token, die im aktuellen Verarbeitungskontext berücksichtigt werden kann.

Dann legen wir einfach noch diese Information dazu!

Stopp, Björn! Mein Tisch hat Grenzen.

Wenn der Tisch voll ist, können keine weiteren Informationen hinzukommen. Es müssen dann Informationen entfernt oder zusammengefasst werden.

CONTEXT WINDOW
(Begrenzte Anzahl an Token)

3

MEMORY – MEHR INFORMATIONEN EXISTIEREN

Memory steht für Informationen, die außerhalb des aktuellen Contexts gespeichert sind und bei Bedarf wieder bereitgestellt werden können.

HAFEN

MANNSCHAFT

ROUTE

EREIGNISSE

KARTEN

NOTIZEN

PLÄNE

UND VIELES MEHR ...

Diese Informationen sind vorhanden, aber nicht automatisch Teil des aktuellen Contexts. Sie können später wieder auf den Tisch gebracht werden.

4

INFORMATIONEN WIEDER BEREITSTELLEN

Eine Anwendung kann bei Bedarf gespeicherte Informationen wieder in den Context bringen.

Super, jetzt haben wir die Information wieder auf dem Tisch.

Jetzt kann ich diese Information wieder berücksichtigen.

HAFEN

5

DAS WICHTIGSTE AUF EINEN BLICK



Context = Informationen, die dem Modell für die aktuelle Verarbeitung zur Verfügung stehen.



Context Window = begrenzte Menge an Token, die im aktuellen Verarbeitungskontext berücksichtigt werden kann.



Memory = Informationen, die außerhalb des aktuellen Contexts gespeichert und bei Bedarf wieder bereitgestellt werden können.

MERKE:

- ✓ Nicht alles, was existiert, liegt auf dem Tisch.
- ✓ Mehr Context ist nicht automatisch besser.
- ✓ Relevante Informationen sind entscheidend.

VERSTEHEN
STATT
RATEN

DER RUNENSTEIN ERINNERT SICH NICHT AN ALLES.
ER ARBEITET MIT DEM, WAS GERADE VOR IHM LIEGT.

KLARE KONZEPTE.
STARKE GRUNDLAGEN.
FÜR DEINE DIGITALE ZUKUNFT.



SI-LLM-03 Context und Memory

1. Einordnung des Moduls

In den ersten beiden Modulen wurde zunächst geklärt, was ein Large Language Model ist und wie es mithilfe der Next-Token-Prediction Text erzeugt. Damit fehlt jedoch noch ein entscheidender Bestandteil: Ein LLM muss nicht nur das nächste Token vorhersagen, sondern dabei auch wissen können, worauf sich die aktuelle Eingabe bezieht.

Wenn ein Benutzer schreibt: „Mein Schiff heißt Sturmfalke“ und anschließend fragt: „Welche Farbe hat es?“, kann die zweite Frage nur sinnvoll beantwortet werden, wenn die vorherige Information bei der Verarbeitung noch zur Verfügung steht. Genau darum geht es in diesem Modul.

Der zentrale Begriff dafür ist Context. Er beschreibt die Informationen, die einem Modell für einen konkreten Verarbeitungsschritt zur Verfügung gestellt werden. Dazu können beispielsweise die aktuelle Eingabe, vorherige Nachrichten, Systemanweisungen, Dokumente oder Ergebnisse von Werkzeugen gehören. Welche Informationen tatsächlich Bestandteil des Contexts sind, hängt von der jeweiligen Anwendung und ihrer technischen Architektur ab.

2. Lernziele

Nach Abschluss des Moduls können die Lernenden den Begriff Context im Zusammenhang mit Large Language Models erklären und von den gelernten Modellparametern unterscheiden. Sie verstehen, welche Rolle Gesprächsverläufe und andere bereitgestellte Informationen bei der Verarbeitung einer Anfrage spielen und können erklären, was mit einem Context Window gemeint ist.

Darüber hinaus können sie beschreiben, warum ein Context Window begrenzt ist, welche Auswirkungen ein sehr umfangreicher Context haben kann und welche technischen Strategien zur Verwaltung längerer Kontexte eingesetzt werden. Sie können außerdem erklären, warum „Memory“ bei LLM-Anwendungen nicht automatisch bedeutet, dass das Modell Informationen dauerhaft gespeichert hat.

3. Didaktische Ausgangslage

Als Analogie begleitet uns weiterhin der uralte graubärtige Orakelwiking aus den vorherigen Modulen. Er steht für das LLM. Der Runenstein ist das Medium, über das die Informationen übermittelt werden. Für dieses Modul betrachten wir jedoch vor allem den Tisch des Orakels.

Auf dem Tisch liegen die Schriftrollen, die der Orakelwiking bei seiner aktuellen Antwort berücksichtigen kann. Im Langhaus können daneben noch viele weitere Schriftrollen liegen. Dass diese Informationen irgendwo vorhanden sind, bedeutet jedoch nicht automatisch, dass sie gerade auf seinem Tisch liegen.

Der Tisch stellt damit eine anschauliche Analogie für den aktuellen Context dar. Seine begrenzte Größe hilft außerdem dabei, das Prinzip des Context Windows zu verstehen.

4. Was bedeutet Context?

Ein Large Language Model verarbeitet eine Anfrage nicht völlig unabhängig von ihrer Umgebung. Die Informationen, die dem Modell für einen konkreten Verarbeitungsschritt zur Verfügung stehen, bilden den

Context. Der Context kann aus unterschiedlichen Quellen stammen und muss nicht ausschließlich aus der Nachricht des Benutzers bestehen.

In einer Chat-Anwendung kann beispielsweise der bisherige Gesprächsverlauf Bestandteil des Contexts sein. Eine Anwendung kann zusätzlich Systemanweisungen, Dokumente, Suchergebnisse oder die Ergebnisse eines aufgerufenen Werkzeugs übergeben. Für das Modell werden diese Informationen zu einem gemeinsamen Verarbeitungskontext.

Das bedeutet nicht, dass das Modell all diese Informationen dauerhaft „merkt“. Sie stehen ihm zunächst lediglich für den jeweiligen Verarbeitungsschritt zur Verfügung.

5. Der Context bestimmt den Zusammenhang

Context ist besonders wichtig, weil einzelne Aussagen ohne ihren Zusammenhang mehrdeutig sein können. Wenn Björn sagt: „Mein Schiff heißt Sturmfalke“, und anschließend fragt: „Wie groß ist es?“, muss das Modell die Information aus der vorherigen Aussage berücksichtigen können, um „es“ sinnvoll auf das Schiff zu beziehen.

Ohne diesen Zusammenhang wäre die zweite Frage wesentlich schwerer zu interpretieren. Der Context liefert also nicht nur zusätzliche Informationen, sondern kann auch bestimmen, wie eine aktuelle Eingabe zu verstehen ist.

Genau deshalb ist Context bei LLM-Anwendungen ein zentraler Bestandteil der Verarbeitung. Das Modell erzeugt seine Antwort nicht nur auf Grundlage des letzten Satzes, sondern auf Grundlage dessen, was ihm im jeweiligen Verarbeitungskontext zur Verfügung steht.

6. Conversation History als Context

Bei einem Chat besteht ein wesentlicher Teil des Contexts häufig aus dem bisherigen Gesprächsverlauf. Dadurch kann ein Modell auf Informationen aus vorherigen Nachrichten Bezug nehmen und beispielsweise eine Rückfrage verstehen, die sich auf eine Aussage mehrere Nachrichten zuvor bezieht.

Dabei ist wichtig, zwischen Gesprächsverlauf und dauerhaftem Gedächtnis zu unterscheiden. Wenn eine Anwendung frühere Nachrichten erneut an das Modell übergibt, stehen diese Informationen im aktuellen Context zur Verfügung. Das bedeutet jedoch nicht, dass das Modell diese Informationen dauerhaft in seinen Modellparametern gespeichert hat.

Wie ein Chatverlauf technisch behandelt wird, hängt von der jeweiligen Anwendung ab. Nachrichten können beispielsweise vollständig weitergegeben, gekürzt oder zusammengefasst werden. Moderne Systeme bieten außerdem Mechanismen zur Verwaltung längerer Kontexte.

7. Das Context Window

Ein Modell kann nicht beliebig viele Informationen gleichzeitig verarbeiten. Die maximale Menge an Token, die innerhalb eines Verarbeitungskontexts berücksichtigt werden kann, wird als Context Window bezeichnet. Die konkrete Größe ist abhängig vom verwendeten Modell und der jeweiligen API.

Für die Analogie bedeutet das: Der Orakelwikingen hat nur einen begrenzten Tisch. Auf diesem Tisch können nur so viele Schriftrollen liegen, dass er sie noch bei seiner aktuellen Aufgabe berücksichtigen kann.

Quelle: [1], [2]

Ein größeres Context Window ermöglicht es grundsätzlich, umfangreichere Eingaben beziehungsweise längere Gesprächsverläufe zu verarbeiten. Es bedeutet jedoch nicht automatisch, dass jede zusätzliche Information hilfreich ist. Mit zunehmender Kontextmenge kann es schwieriger werden, relevante Informationen zuverlässig zu nutzen.

8. Wenn der Tisch voll ist

Wird ein Context zu groß, muss die Anwendung damit umgehen. Welche Strategie eingesetzt wird, hängt von der jeweiligen Architektur ab. Nicht benötigte Informationen können entfernt, ältere Gesprächsteile zusammengefasst oder relevante Informationen gezielt erneut bereitgestellt werden.

Auch die Aufteilung großer Eingaben in kleinere Abschnitte oder eine Vorverarbeitung kann sinnvoll sein. OpenAI nennt unter anderem das Kürzen von Prompts, das Entfernen unnötiger oder wiederholter Informationen, das Aufteilen großer Eingaben sowie das Zusammenfassen oder Vorverarbeiten von Text als mögliche Strategien, wenn eine Eingabe zu groß wird.

Quelle: [1], [2]

In der Analogie werden also nicht einfach alle Schriftrollen immer weiter auf dem Tisch gestapelt. Stattdessen muss jemand entscheiden, welche Informationen für die aktuelle Aufgabe tatsächlich relevant sind.

9. Context ist nicht das Modellwissen

Eine der wichtigsten Unterscheidungen dieses Moduls lautet: Context und Modellwissen sind nicht dasselbe.

Das Wissen, das ein Modell während seines Trainings gelernt hat, ist Bestandteil seiner Modellparameter. Der Context hingegen besteht aus Informationen, die dem Modell für einen konkreten Verarbeitungsschritt zur Verfügung gestellt werden. Das Context Window ist daher vom Datenbestand zu unterscheiden, auf dem das Modell trainiert wurde.

Quelle: [2]

Die Analogie mit den Schriftrollen darf deshalb nicht so verstanden werden, als befände sich das gesamte Wissen des LLM in einer riesigen Bibliothek, aus der es bei jeder Frage eine passende Schriftrolle zieht. Die Schriftrollen stellen nur Informationen dar, die der Anwendung aktuell als Context zur Verfügung gestellt werden.

10. Was bedeutet „Memory“?

Der Begriff Memory ist bei LLM-Anwendungen nicht eindeutig. Er kann unterschiedliche technische Mechanismen beschreiben. Eine Unterhaltung kann beispielsweise dadurch fortgeführt werden, dass frühere Nachrichten erneut in den Context aufgenommen werden. Eine Anwendung kann zusätzlich Informationen dauerhaft in einer Datenbank speichern und diese später wieder abrufen.

Damit entsteht eine wichtige Unterscheidung: Das Modell selbst muss keine dauerhafte Erinnerung besitzen, damit eine Anwendung eine Memory-Funktion anbieten kann. Die Anwendung kann gespeicherte Informationen bei Bedarf wieder in den Context einfügen.

Für die Ausbildung ist deshalb die Formulierung „Das LLM erinnert sich“ häufig zu ungenau. Technisch interessanter ist die Frage: Wo wurde die Information gespeichert und wie gelangt sie bei einer späteren Anfrage wieder in den Context?

11. Kurzzeitkontext und externe Erinnerung

Das Context Window lässt sich didaktisch mit einem begrenzten Arbeitsgedächtnis vergleichen. Der Context beschreibt die Informationen, die dem Modell für die Verarbeitung übergeben werden.

Eine Anwendung kann darüber hinaus externe Speicher verwenden. Dort können beispielsweise Gesprächsdaten, Benutzerinformationen, Dokumente oder andere strukturierte Informationen gespeichert werden. Werden diese Informationen später benötigt, kann die Anwendung die passenden Daten auswählen und erneut als Context bereitstellen.

Quelle: [2], [3]

Damit entsteht ein Zusammenspiel aus Speichern, Auswählen und Bereitstellen. Das eigentliche Modell verarbeitet anschließend den Context, der ihm für die aktuelle Anfrage übergeben wurde.

12. Externer Speicher

Externe Speicher können technisch sehr unterschiedlich umgesetzt werden. Je nach Anwendungsfall kommen beispielsweise relationale Datenbanken, Dokumentenspeicher, Vektordatenbanken, Dateisysteme oder spezielle Knowledge Bases zum Einsatz.

Entscheidend ist dabei nicht der konkrete Speichertyp, sondern die Trennung zwischen Speicher und Context. Eine Information kann dauerhaft in einem Speichersystem vorhanden sein, ohne Bestandteil der aktuellen Anfrage zu sein.

Dieses Prinzip wird später bei RAG (Retrieval-Augmented Generation) besonders wichtig. Dort werden relevante Informationen aus externen Quellen abgerufen und dem Modell als zusätzlicher Context zur Verfügung gestellt. Die konkrete Funktionsweise von RAG wird in SI-LLM-06 ausführlich behandelt.

13. Context Management

Sobald eine LLM-Anwendung mit längeren Gesprächen, Dokumenten oder mehreren Werkzeugen arbeitet, wird die Verwaltung des Contexts zu einer Aufgabe der Systemarchitektur.

Die Anwendung muss entscheiden, welche Informationen erhalten bleiben, welche entfernt werden können und welche bei Bedarf erneut abgerufen werden müssen. Dabei spielen nicht nur die maximale Context-Größe, sondern auch Relevanz, Datenmenge, Verarbeitungskosten und Antwortzeit eine Rolle.

Gerade für die Systemintegration ist dieser Bereich interessant: Context Management ist keine rein sprachliche Eigenschaft des LLM, sondern Bestandteil der Infrastruktur und der Anwendungsarchitektur.

14. Viel Context bedeutet nicht automatisch besseren Context

Ein häufiger Denkfehler besteht darin, möglichst viele Informationen an ein Modell zu übergeben und davon auszugehen, dass eine größere Informationsmenge automatisch zu einer besseren Antwort führt.

Das ist nicht zwingend der Fall. Zusätzliche Informationen können irrelevant, redundant oder sogar widersprüchlich sein. Je größer der Context wird, desto wichtiger wird daher die Auswahl relevanter Informationen.

Quelle: [2], [3]

Die technische Aufgabe lautet deshalb nicht einfach „möglichst viel Context bereitstellen“, sondern „den für die aktuelle Aufgabe relevanten Context bereitstellen“.

15. Context und Token

Da LLMs Text in Token verarbeiten, wird auch der Context letztlich in Token gemessen. Die Anzahl der Token entspricht dabei nicht einfach der Anzahl der Wörter. Ein Token kann beispielsweise ein Zeichen, ein Wortbestandteil, ein vollständiges Wort oder ein Satzzeichen darstellen. Außerdem kann derselbe Text je nach Modell, Tokenizer, Sprache und Codierung unterschiedlich tokenisiert werden.

Quelle: [1]

Für die Systemintegration ist diese Eigenschaft relevant, weil Context-Größen, API-Nutzung und teilweise auch Kosten auf Tokenmengen basieren. Ein scheinbar kurzer Text kann daher eine andere Tokenmenge erzeugen als ein gleich langer Text in einer anderen Sprache oder mit einer anderen Tokenisierung.

16. Was passiert bei einer langen Unterhaltung?

Mit jeder weiteren Nachricht kann der Gesprächskontext wachsen. Wenn die Anwendung den bisherigen Verlauf vollständig weitergibt, steigt damit auch die Menge der zu verarbeitenden Token.

Irgendwann kann die verfügbare Context-Kapazität erreicht werden. Dann muss die Anwendung eine Strategie zur Context-Verwaltung einsetzen. Je nach System kann sie beispielsweise ältere Inhalte entfernen, Nachrichten zusammenfassen oder nur relevante Informationen weitergeben. Auch serverseitige Verfahren zur automatischen Verdichtung längerer Gesprächsverläufe werden eingesetzt.

Quelle: [2], [3]

Für den Benutzer kann das Verhalten wie ein „Vergessen“ aussehen. Technisch kann jedoch etwas ganz anderes passiert sein: Die betreffende Information befindet sich schlicht nicht mehr im Context, der für die aktuelle Verarbeitung verwendet wird.

17. Context und Inferenz

Der Zusammenhang mit dem vorherigen Modul lässt sich nun vollständig herstellen. In SI-LLM-02 wurde gezeigt, dass ein LLM nicht die komplette Antwort auf einmal erzeugt, sondern Token für Token vorhersagt.

Für diese Vorhersage benötigt das Modell einen Context. Dieser Context beeinflusst, welche Token als mögliche Fortsetzung betrachtet werden. Das ausgewählte Token wird anschließend Teil der erzeugten Ausgabe und kann bei der weiteren Generierung wiederum zum Verarbeitungskontext gehören.

Vereinfacht ergibt sich damit die Kette:

Eingabe → Tokenisierung → Context → LLM → Next-Token-Prediction → neues Token → aktualisierter Context

Damit bildet SI-LLM-03 die Brücke zwischen der Token-basierten Texterzeugung aus Modul 2 und den später behandelten technischen Komponenten einer LLM-Anwendung.

18. Praxisbezug Systemintegration

Für Fachinformatiker der Fachrichtung Systemintegration ist Context nicht nur ein theoretisches Modellthema. Sobald ein LLM in eine reale IT-Infrastruktur integriert wird, entstehen konkrete technische Anforderungen.

Dazu gehören beispielsweise die Verarbeitung großer Datenmengen, die Verwaltung von Gesprächsverläufen, die Bereitstellung externer Wissensquellen, die Skalierung von Anwendungen, Latenzzeiten und Ressourcenverbrauch. Auch Datenschutz und Datenhaltung spielen eine Rolle, weil Informationen aus Gesprächen, Dokumenten oder externen Systemen Bestandteil eines Contexts werden können.

Ein Administrator oder Infrastrukturplaner muss deshalb nicht nur wissen, welches Modell verwendet wird. Er muss auch verstehen, welche Daten an das Modell übergeben werden, wo diese Daten gespeichert werden, wie lange sie vorgehalten werden und welche Komponenten zwischen Benutzer und Modell für die Zusammenstellung des Contexts verantwortlich sind.

19. Typische Denkfehler

„Context ist das Gedächtnis des LLM.“ Diese Aussage ist als Vereinfachung verständlich, technisch aber zu ungenau. Context bezeichnet die Informationen, die dem Modell für einen Verarbeitungsschritt zur Verfügung stehen.

„Alles, was das Modell einmal gesehen hat, kann es später wieder abrufen.“ Daraus folgt nicht automatisch eine dauerhafte Erinnerung. Trainingswissen, aktueller Context und externe Speicher müssen voneinander unterschieden werden.

„Das LLM hat eine Datenbank mit Antworten.“ Ein LLM ist nicht einfach eine Datenbank, aus der bei jeder Anfrage ein passender Datensatz gelesen wird. Seine Modellparameter repräsentieren während des Trainings erlernte Strukturen und Zusammenhänge.

„Je größer der Context, desto besser die Antwort.“ Ein größerer Context kann mehr Informationen aufnehmen, aber irrelevante oder redundante Informationen können die Verarbeitung erschweren.

„Wenn eine Information gespeichert ist, kennt das LLM sie.“ Eine gespeicherte Information muss dem Modell für die betreffende Anfrage erst zugänglich gemacht werden, beispielsweise indem sie in den Context aufgenommen wird.

20. Zusammenfassung

Context bezeichnet die Informationen, die einem LLM für einen konkreten Verarbeitungsschritt zur Verfügung stehen. Dazu können neben der aktuellen Eingabe auch vorherige Nachrichten, Systemanweisungen, Dokumente oder Ergebnisse externer Werkzeuge gehören. Die konkrete Zusammensetzung hängt von der jeweiligen Anwendung ab.

Das Context Window begrenzt die Menge an Token, die ein Modell innerhalb eines solchen Verarbeitungskontexts berücksichtigen kann. Wird der Context zu groß, muss die Anwendung geeignete Strategien zur Verwaltung der Informationen einsetzen. Dazu gehören unter anderem Kürzen, Zusammenfassen, Filtern und das erneute Abrufen relevanter Informationen.

Memory ist davon zu unterscheiden. Eine Anwendung kann Informationen dauerhaft außerhalb des Modells speichern und sie später wieder in den Context einfügen. Damit wird deutlich, dass „Gedächtnis“ bei LLM-Anwendungen häufig eine Eigenschaft des Gesamtsystems und nicht einfach eine Eigenschaft des Modells selbst ist.

21. MERKE

Context ist das, was dem Modell für die aktuelle Verarbeitung zur Verfügung steht. Das Context Window begrenzt, wie viele Token dabei gleichzeitig berücksichtigt werden können. Memory kann durch zusätzliche Systemkomponenten bereitgestellt werden. Was irgendwo gespeichert ist, ist deshalb nicht automatisch Bestandteil des aktuellen Contexts.

Beispiel: Was gehört zum Context?

Stell dir eine einfache Chat-Anwendung vor. Ein Benutzer schreibt:

Benutzer: Mein Server heißt SV-FILE01 und läuft mit Debian 13.

Die Anwendung übergibt diese Nachricht an das LLM. Sie ist damit Bestandteil des aktuellen Contexts.

Anschließend fragt der Benutzer: „Welches Betriebssystem verwendet der Server?“ Die Anwendung übergibt den bisherigen Gesprächsverlauf zusammen mit der neuen Frage:

Benutzer: Mein Server heißt SV-FILE01 und läuft mit Debian 13.

Benutzer: Welches Betriebssystem verwendet der Server?

Das LLM kann nun erkennen, dass sich „der Server“ auf SV-FILE01 bezieht und dass als Betriebssystem Debian 13 genannt wurde.

Würde die Anwendung dagegen ausschließlich die zweite Nachricht an das Modell übergeben, fehlt diese Information im aktuellen Context. Das Modell hat dann keinen direkten Zugriff auf die vorherige Aussage und kann die Frage deshalb nicht zuverlässig anhand dieser Information beantworten.

Wichtig: Die erste Nachricht wurde durch die Unterhaltung nicht automatisch zu dauerhaftem Modellwissen. Sie war zunächst einfach Bestandteil des Contexts, den die Anwendung bei der zweiten Anfrage erneut an das Modell übergeben hat.

Technischer Ablauf

1. Benutzer sendet Nachricht
↓
2. Anwendung speichert/verwaltet den Gesprächsverlauf
↓
3. Anwendung erstellt den Context
↓
4. Context wird tokenisiert
↓
5. LLM verarbeitet den Context
↓
6. LLM erzeugt die Antwort

Quellen

[1] OpenAI Help Center – Token verstehen und zählen

<https://help.openai.com/de-de/articles/4936856-was-sind-tokens-und-wie-z%C3%A4hlt-man-sie>

[2] Anthropic – Claude Platform Docs: Context windows

<https://platform.claude.com/docs/de/build-with-claude/context-windows>

[3] Google AI for Developers – Long context

<https://ai.google.dev/gemini-api/docs/long-context>